

CPT205 – Computer Graphics

Assessment 2 – 3D Modelling Project

Sunxi_Peng-2361598

Information and Computing Science

1.Overview

This program presents a cyber-style futuristic city in 3D using OpenGL and FreeGLUT, integrating key graphics techniques such as texture mapping, lighting, projection, hierarchical modeling, and dynamic animation. The scene is centered on a central energy tower, with major roads extending along the X and Z axes and three concentric rings of high-rise buildings surrounding it. In the aerial layer, dual-loop sky rails carry suspended light-rail trains, accompanied by floating futuristic vehicles. Users can navigate the city by controlling a robot character via keyboard input. The entire environment is enclosed within a textured spherical sky dome that displays a starfield. Overall, the system uses animation to highlight the city's technological character and applies a variety of cyber-themed textures to enhance the visual quality of the futuristic environment.

2.Design Concept

The 3D futuristic city follows a radial design, forming a multi-layered urban structure centered on the core tower, with major arterial roads defining the overall spatial layout.

The central core tower is constructed from simple geometric primitives—cylinders, cones, and spheres. A rotating energy sphere at the top represents the city's power source. The tower's base connects directly to the north-south main roads, where dynamic "breathing lights" change color over time, adding to the city's futuristic, energy-driven aesthetic.

The ground and high-rise building clusters further define the environment. A cyber-themed texture is applied to the ground to enhance the modern aesthetic, while the buildings are arranged into three concentric rings around the core tower. The textured skyscrapers add richness and depth to the city's overall visual presentation.

In the aerial section of the scene, two circular sky rails connect the inner and outer building rings, with trains—constructed from simple geometric shapes—traveling smoothly along the tracks. At a lower altitude, rectangular floating cars move between buildings, adding another layer of motion to the city.

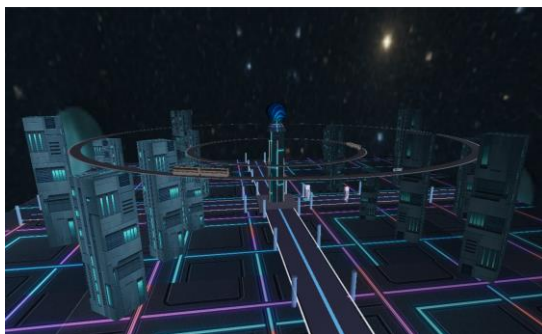


Figure1: Demonstration image of the futuristic city



Figure2: Different angle of the city

3. Graphics Features

We now proceed to explain the key graphics techniques employed in this project and how they were implemented.

3.1 Basic Geometric Primitives Design

In this project, all objects in the city—including buildings, trains, and other structures are constructed from simple geometric primitives. At the beginning of the code, these basic shapes are defined and rendered. Within each drawing function, `glRotatef` is used to adjust the object's orientation, while GLU functions such as `gluCylinder` generate the actual geometric surfaces. These routines are then encapsulated into reusable functions, allowing the shapes to be invoked efficiently throughout the program.

Cylinders form the main structure of the core tower, while cones and spheres shape the energy sphere at its top. Cubes are used to build the surrounding buildings. Other geometric elements are generated directly within their respective rendering functions.

3.2 Viewpoint Control Design

After the geometric primitives are rendered, they are arranged within the city layout and prepared for viewing. The program uses `gluLookAt` to keep the camera focused on the city center, allowing users to rotate around it via keyboard input. A 60-degree `gluPerspective` projection is applied to create a realistic sense of depth, where closer objects appear larger and distant ones appear smaller.

3.3 City Layout Design

The city's core tower is positioned at the absolute center of the scene, at coordinates (0, 0, 0). Extending from its base is a cross-shaped road network aligned along the X and Z axes. On both sides of these roads, sci-fi light strips are implemented using elongated cubes created through `glScalef`. Their colors change dynamically over time using a sine-based function, adding a strong technological and futuristic atmosphere to the city.

For the building clusters, the `drawBuildingRing()` function divides the structures into three concentric rings by using different radii. By varying the angle in increments of π and related values, the buildings are placed evenly along circular paths. Their orientations are then adjusted with `glRotatef` so that all buildings consistently face toward the central core tower.

In the aerial section, `drawRailCircle()` controls the radius and height of the sky rails, while `drawRailAndTrain()` uses trigonometric functions to animate the trains along circular trajectories. The floating cars, modeled from cubes, travel along the X and Z axes. The motion of the trains is driven by updates to `trainAngle`, which produces continuous circular movement. In contrast, the floating cars move forward by incrementing their coordinates and incorporate slight vertical oscillation to simulate

hovering.

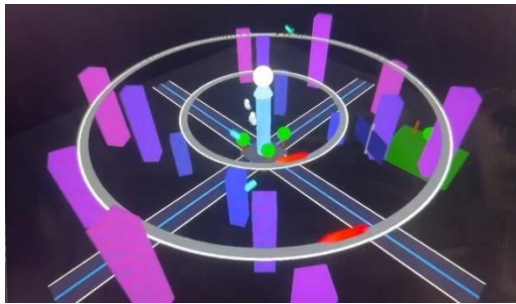


Figure3: Simple City Design Diagram

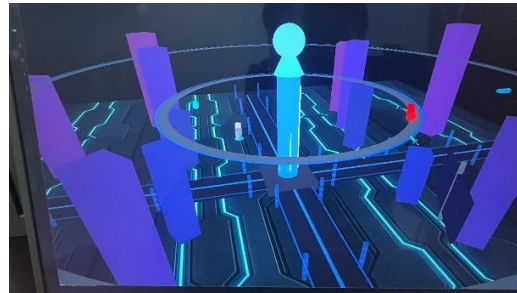


Figure4: Experiment with Different Ground Textures

3.4 Texture Mapping Design

With only basic colors and layouts, the initial city appears flat and lacks a futuristic technological feel, as shown in Figure 3. To enhance the visual quality, the program introduces texture mapping with selected texture images. Drawing from Lab 11, the program uses `loadTextureBMP` to read BMP images, parse their data, and upload the textures to OpenGL. During object rendering, the corresponding texture is bound to each model. For example, the ground texture is applied by calling `glBindTexture(GL_TEXTURE_2D, groundTex)` and assigning appropriate texture coordinates. The tiling factor is set to 4 to increase clarity, resulting in the effect shown in Figure 4. For curved surfaces, texture mapping is handled using `gluQuadricTexture`, which ensures smooth and continuous texture distribution. An example is the outer sky sphere of the scene, where a starfield texture is applied.

3.5 Lighting and Materials Design

After applying textures, the city begins to convey a futuristic atmosphere; however, at this stage, the objects still lack defined materials and lighting. To achieve a more realistic and technologically immersive visual effect, the program introduces a point light source (`GL_LIGHT0`) using `GLfloat lightPos[]`, positioned above the scene to cast light downward and create visible shadowing. Smooth shading is enabled to produce natural light transitions across building surfaces, resulting in clearer highlights and gradients.

For material configuration, different objects use reflection parameters suited to their function and real-world counterparts. The core energy tower is assigned a high specular coefficient to create a metallic, glossy appearance, while skyscrapers use moderate shininess to produce a subtle reflective effect. In contrast, the road surface is given weak diffuse reflection to better match realistic pavement behavior. To maintain consistent visuals, the sky sphere is excluded from lighting calculations so it remains at a constant brightness. With `GL_COLOR_MATERIAL` enabled, ambient and specular responses can be controlled directly through `glColor3f()`, simplifying the process of assigning material properties during rendering.

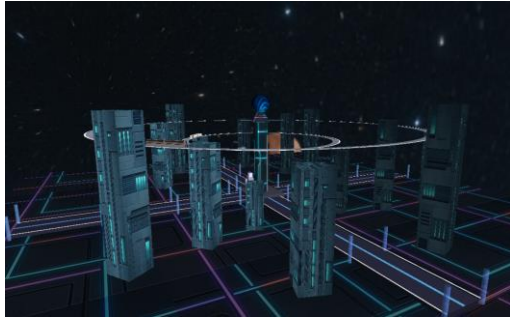


Figure 5: Lighting and Material Visualization



Figure 6: Preview of the user-controlled robot

3.6 User Robot Design

To improve user experience, the program introduces a user-controlled robot. The robot model is built using hierarchical modeling and consists of multiple geometric subcomponents organized under a unified body structure, ensuring coordinated movement. Users can control the robot via keyboard input to navigate through the futuristic city, enhancing immersion and interaction within the scene. As shown in Figure 6, the user-controlled robot is visible within the scene.

4. User Instructions:

The program is simple to operate. As long as the executable and texture files are kept in the same directory, the program can be launched by double-clicking the executable. The keys W, A, S, and D control the movement and rotation of the user-controlled robot. The J and L keys adjust the viewing direction, while I and K change the camera distance from the city center. Additionally, Page Up and Page Down modify the camera height. With these controls, users can freely explore and experience the futuristic city from different perspectives.

5. Conclusion

This project demonstrates how OpenGL and FreeGLUT can be used together to construct a futuristic 3D city environment. Beginning with basic geometric modeling, the system incrementally integrates hierarchical modeling, texture mapping, lighting and material configuration, view control, and dynamic animation, resulting in an interactive virtual city. While the program functions as intended, several limitations remain. Not all objects have fully defined material properties, and lighting effects such as functional streetlights along the road have not yet been implemented. Additionally, the model complexity and level of detail are still relatively limited. Looking ahead, I plan to further refine and expand the city using the graphics concepts learned throughout this course, with the goal of improving both visual fidelity and functional depth.